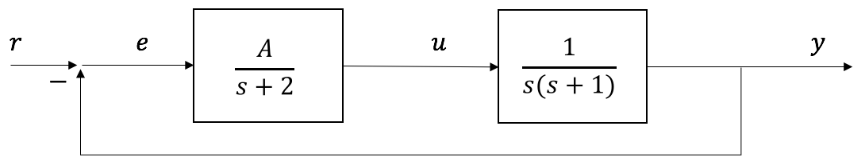
# Homework 7

## Design Challenges

The following homework consists of 4 design challenges of varying levels of difficulty. I suggest completing all required steps before attempting any optional bonus problems.

**Problem 1**
Consider the following block diagram:




a) Determine the loop gain of the system.
b) Determine the system type.
c) Determine the steady state error due to step, ramp, and parabolic inputs as functions of $A$.
d) Consider a unit step input. How does the % overshoot vary as a function of $A$? Use a root locus plot to aid your explanation (you can use rlocus in Matlab – remember the equation for the poles must be in root locus form). Also show the simulation output at 2 values of $A$.
e) For what values of $A$ is the system stable? Use a root locus plot or Routh array to aid your explanation (you can use rlocus() or routh() in Matlab).

**Problem 2**
The loop gain in a servomotor control system is given by

$$L(s) = \frac{K}{s(s+\alpha)}$$

f) Assuming unity feedback, pick values of $K$ and $\alpha$ such that
   (i) the steady state error due to a ramp $e_{ss,ramp} = 0.04$.
   (ii) the % overshoot due to a unit step is 16.3%.
g) Simulate the system with step and ramp inputs and show that your parameters meet the specifications.

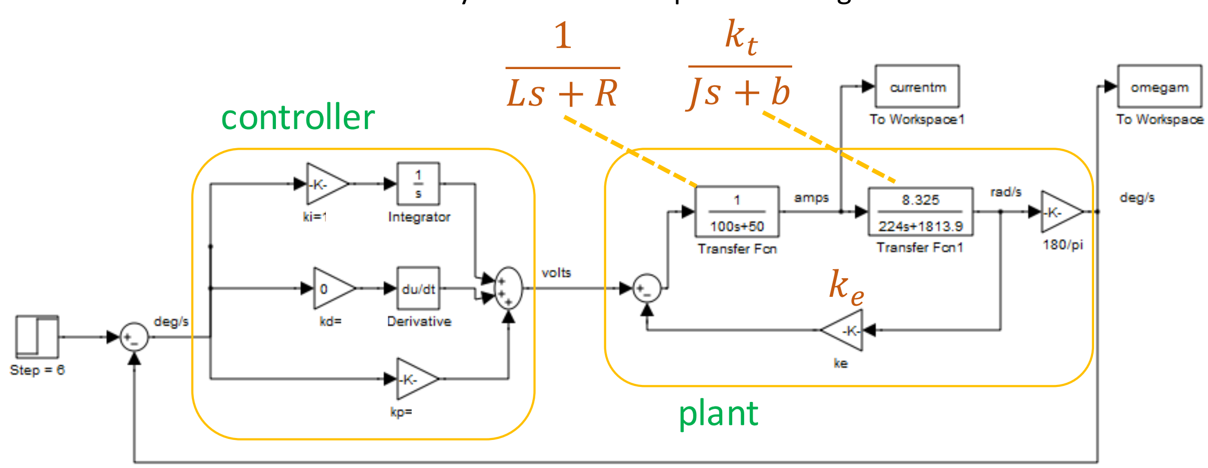
**Problem 3 – Electromechanical design**

Read Section 4.3 of the book for background information on PID control and tuning. Pay particular attention to Section 4.3.6 "Ziegler-Nichols Tuning" for one way of how to select PID coefficients. Although these coefficients will need to be revised based on the specifications you want to meet, they usually will be good initial guesses. Note that for some systems (hint: possibly in this case), you can increase the gain to infinity and the system will never become unstable – in this case, the ultimate sensitivity method of tuning will fail, and you must use a different method. There is no general purpose PID tuning method that applies to all systems without iteration / optimization.

An antenna subsystem has the parameters shown in the table below. The input to the system is the motor voltage (volts). The outputs are the angular velocity (deg/s) and current (amps).

| System | |
|---|---|
| J | 224 kg $m^2$ |
| b | 1813.9 Nms |
| ke | 8.325 Nm/A |
| kt | 8.325 Nm/A |
| R | 50 Ω |
| L | 100 H |

PID feedback is used to control the system. An example block diagram is shown below.




The system has the following requirements

| Type | Requirement |
|---|---|
| steady state error for step | $e_{ss} = 0$ |
| 1% settling time to a step | $ts_{,1\%} < 2.8$ s |
| current maximum for a step input of 6 deg/s (not unit step) | $I_{max} < 30$ A |
| % overshoot Mp due to a step | Mp < 40% |

h) Recreate the system in Simulink. Note: use the PID controller block instead of trying to create a PID controller manually if you plan on using derivative control (the du/dt block does not handle step inputs very well).

i) Start with a proportional controller $C(s) = k_P$. Is it possible to meet all of the requirements? If so, provide output plots at that value of $k_P$. If not, prove this using root locus or another method.

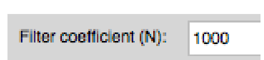
j) Consider a PI controller ($C(s) = \frac{k_I}{s} + k_P$). Select controller values of $k_P$ and $k_I$ that satisfy all of the requirements. Prove the requirements are satisfied either analytically or through simulation. Explain your methodology.

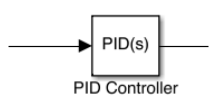
k) Fill out the table below:

| Requirement type | Requirement | Actual | | |
|---|---|---|---|---|
| | | P<br>$k_P =$ | PI<br>$k_P =$<br>$k_I =$ | PID<br>$k_P =$<br>$k_I =$<br>$k_D =$ |
| steady state error for step | $e_{ss} = 0$ | | | |
| 1% settling time to a step | $t_{s,1\%} < 2.8$ s | | | |
| current maximum for a step input of 6 deg/s (not unit step) | $I_{max} < 30$ A | | | |
| % overshoot Mp due to a step | Mp < 40% | | | |
| rise time | tr (no requirement) | | | |

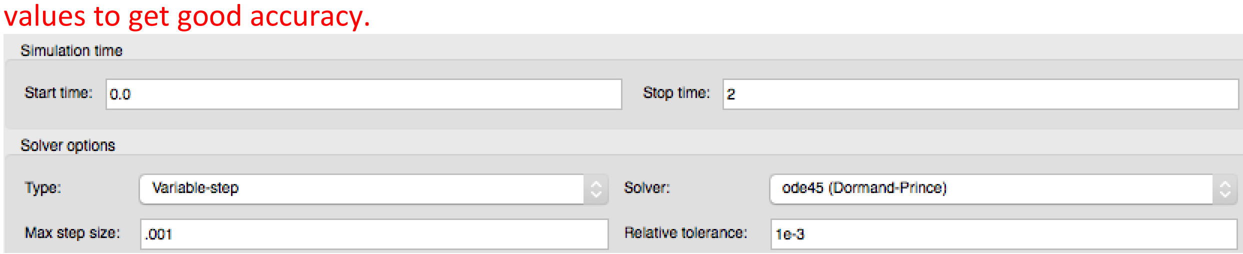
* highlighted cells are for the bonus problem

l) **Optional bonus:** Consider a PID controller, where $k_D$ may be negative (which would mean removing damping from the system). Select PID coefficients such that all of the requirements are met while optimizing for the fastest rise time. Explain your methodology.
Note: use the PID controller block instead of trying to create a PID controller manually if you plan on using derivative control (the du/dt block does not handle step inputs very well). In this block, make the filter coefficient very large to approximate ideal derivatives.



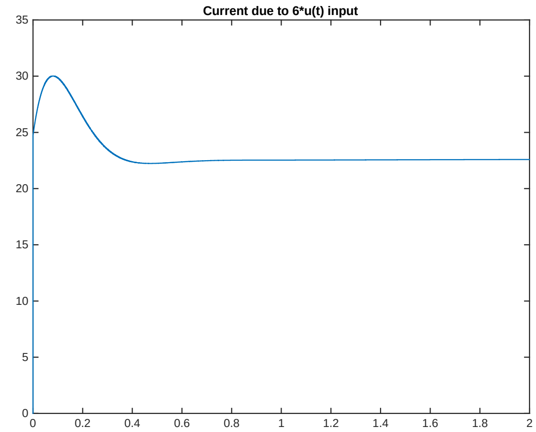



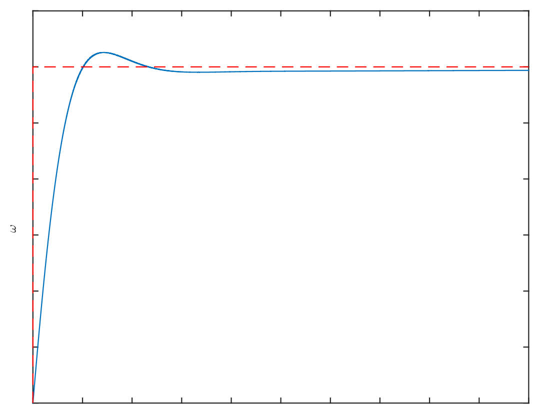

Make sure you change the simulation max step time and stop time to appropriate values to get good accuracy.



If optimizing programmatically, it is much easier using Matlab transfer functions with stepinfo() rather than using Simulink.

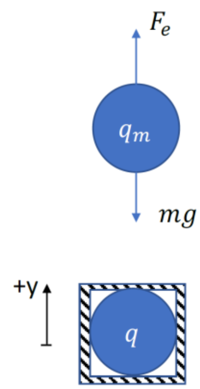
My optimal solution has a rise time of 0.1399 seconds. Can you beat it?



## Problem 4 – Electrostatic Levitation

Consider 2 charged particles of charges $q$ and $q_m$, one that is fixed and the other that is free to move. The movable change is perfectly vertical above the fixed charge:




The electrostatic force is given by Coulomb's law:

$$F_e = \frac{kqq_m}{r^2}$$

where $k = 9 * 10^9 \mathrm{Nm}^2/\mathrm{C}^2$ and $r$ is the distance between the particles (in this case, $r = y$). If $q$ and $q_m$ have the same signs of charges, the force is repulsive, while if they have opposite signs, the force is attractive.

m) Write a symbolic expression for the equilibrium distance ($y_{ss}$) between the particles assuming both charges are positive.

n) For $q = q_m = 2 * 10^{-3}$ C and $m = 100$ kg, calculate the steady state position and show that if you initialize the simulation ("ElectrostaticLevitation_1D.m") with these values and this position, the levitating mass will not move. For example, if you calculated $y_{ss} =$ 4,

```
rm0 = [0 4]; % (x, y) initial position of levitating object [m]
```

o) Is this a linear, time invariant system (LTI)? Why or why not?

p) Calculate the natural frequency and period of the system about the steady-state position calculated previously. Hint: you can calculate the effective stiffness using

$$k_{eff} = \left.\frac{\partial F_e}{\partial y}\right|_{y=y_{ss}}$$

Then, the natural frequency is

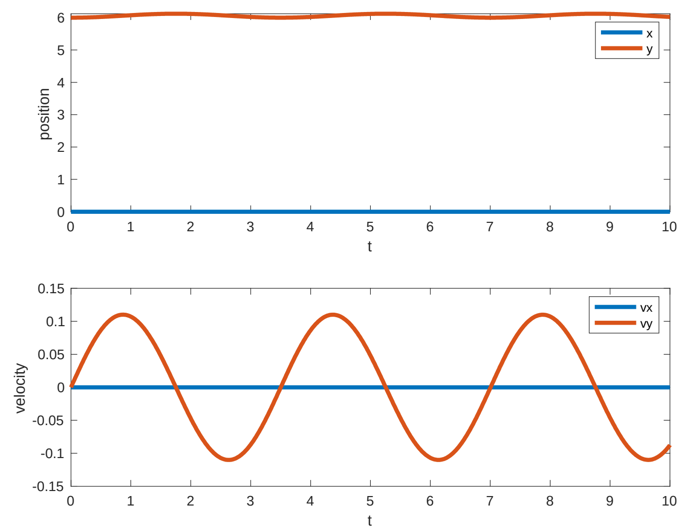
$$\omega_n = \sqrt{\frac{k_{eff}}{m}}$$

Alternatively, you can cast the system in linearized state space form and examine the equation that represents a standard harmonic oscillator:

$$\ddot{y} = -\omega_n^2 y$$

q) Initialize the position of the mass very close, but not exactly at steady-state.

Show the levitating mass oscillates at the frequency you calculated. Hint: your plots should look something like this:



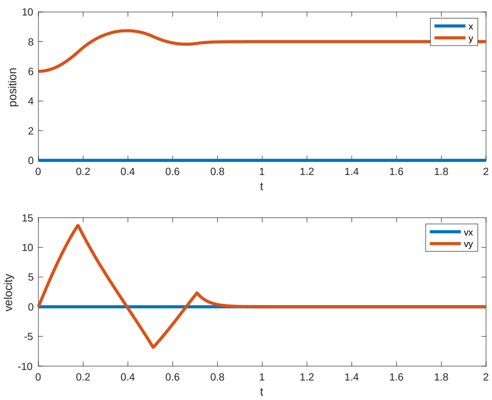
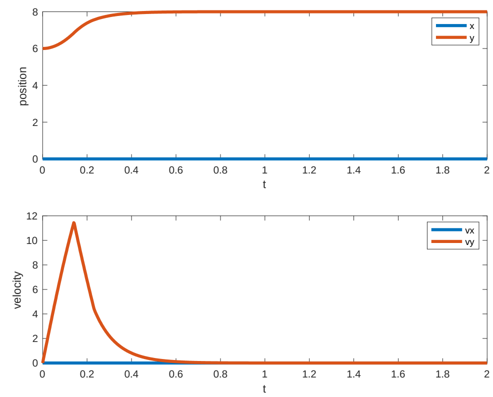

r) Based on the natural frequency you calculated, what are the poles of the linearized system?

s) In this case, P feedback alone is not sufficient because we need to add damping to the system. Design a PD controller that allows you to manipulate the location of the levitating mass by changing the charge $q$. Note that charge limits $|q| < 20 * 10^{-3}\ \mathrm{C}$ have been imposed. The controller code is all located in the function

```
function q_new = Controller(q0, r, rm, vm, t)
```

Start the system at steady-state. Say you want the system to go to $y = 8$, which represents a step input. Run the simulation, and show the output. Explain how you tuned the P and D coefficients to obtain fast settling times (<1 second) with no overshoot ($\zeta \geq 1$)

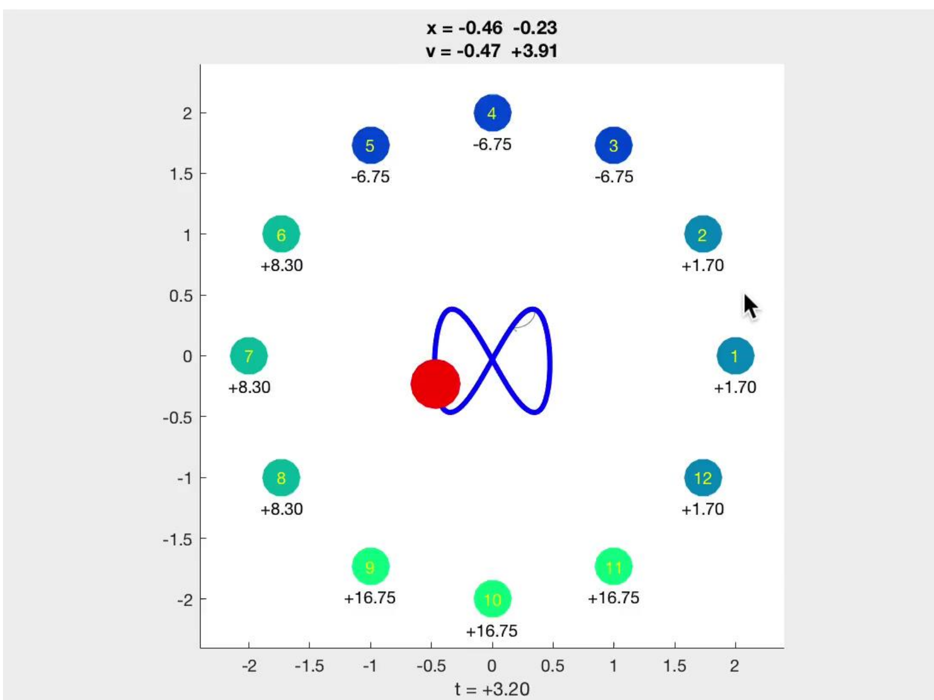
Example outputs for 2 PD controllers are shown below:

| Underdamped tuning | Overdamped tuning |
|---|---|
|  |  |

t) **Optional bonus:** Consider a charge cage, where fixed charges are placed every 30 degrees. See code “ElectrostaticLevitation_3D.m”, which is almost identical to the previous version of the code, but has more fixed charges initialized.

Design a PD controller to manipulate the levitating particle in 3D by varying the charges of all fixed points independently.

- Show that for a step input (which would represent specifying a x and y position for the charge to move to), your scheme works and is fast with minimal overshoot.
- Show you can follow specific paths in space, for example an infinity symbol:




See link for animation:
https://drive.google.com/file/d/1Y9WKhwLoqgrBInxhEnufEB1BWTPBAmUz/view?usp=sharing